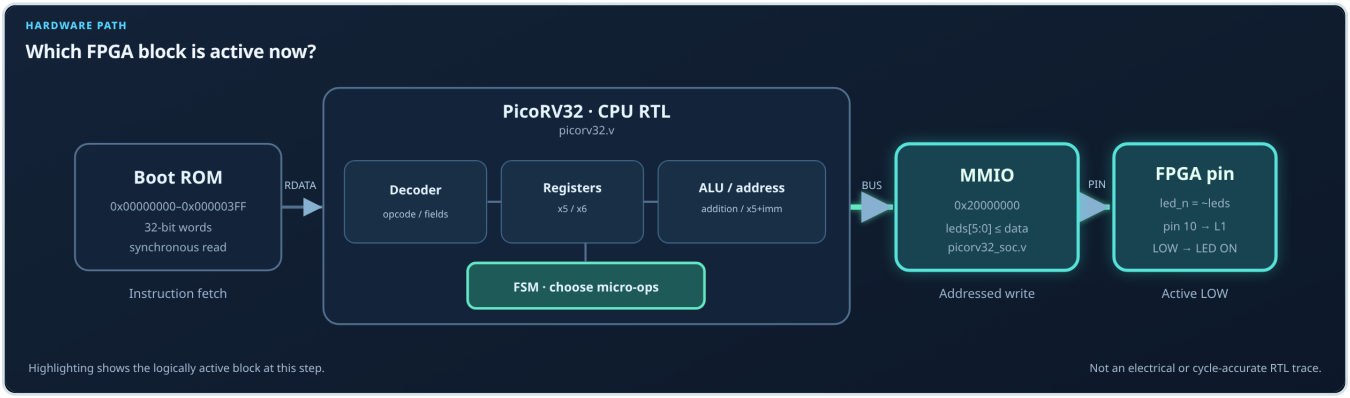
# From Machine Code to a Glowing LED

A fully illustrated, 19-step companion to the interactive PicoRV32 / RV32I HTML demonstration on Tang Nano 9K

19 conceptual steps | 4 RV32I instructions | ROM → CPU → MMIO → LED




Actual English demo block diagram: the MMIO register drives an active-LOW FPGA pin.

**A. A. Malachevsky**

Independent researcher · ORCID 0009-0008-6009-3196
Project: github.com/AIDevelopersMonster/Sipeed-FPGA

Companion to `PicoRV32_LED1_Interactive_Demo_EN.html` · document v1.0

GETTING STARTED

# How to use the interactive demo

Open **PicoRV32_LED1_Interactive_Demo_EN.html** in a modern browser by double-clicking the file. No server, Internet connection, compiler, or physical FPGA board is required. Choose **L1**, press **“↻ Reset”**, and proceed with **“Step →”**.




The working English interface: select L1–L6, step backward/forward, Auto, Jump to ON, Reset and speed.

## Controls

- **Step →** moves to the next conceptual event.
- **← Back** returns to the previous event.
- **▶ Auto** advances automatically at the selected speed.
- **Jump to ON ↗** skips to the acknowledged SW transaction (step 14).
- **↻ Reset** restores the initial state.
- **L1–L6** selects the LED and updates the ADDI immediate.

## Five synchronized views

- **Program:** conceptual C and four assembly lines.
- **Machine word:** 32 bits, opcode, immediate and encoding.
- **CPU:** PC, x5, x6, fetched instruction.
- **Memory interface:** valid/ready, address, data, strobes.
- **LEDs:** stored logical value and active-LOW pins.

> Reading rule: **one HTML step is not necessarily one PicoRV32 clock cycle.** The demo visualizes the meaning of operations, not the timing of a real RTL simulation.

PROGRAM AND MEMORY

# Four ROM words: the complete demonstration program

In C the intended operation is `*(volatile uint32_t*)0x20000000 = 1;`. The HTML does **not** claim that a given C compiler emits exactly these four instructions. The following purpose-built RV32I program is used instead.

```
_start:
    lui  t0, 0x20000
    addi t1, zero, 1
    sw   t1, 0(t0)
stop:
    jal  zero, stop
```

| Byte address | Machine word | Assembly | Little-endian bytes |
|---|---|---|---|
| 0x00000000 | **0x200002B7** | LUI x5,0x20000 | B7 02 00 20 |
| 0x00000004 | 0x00100313 | ADDI x6,x0,1 | 13 03 10 00 |
| 0x00000008 | 0x0062A023 | SW x6,0(x5) | 23 A0 62 00 |
| 0x0000000C | 0x0000006F | JAL x0,0 | 6F 00 00 00 |

### Where does 0x200002B7 come from?

For U-type, bits [31:12] contain `0x20000`, bits [11:7] encode `rd=5`, and bits [6:0] encode `0110111` (LUI). The result is `0x20000000` in x5.

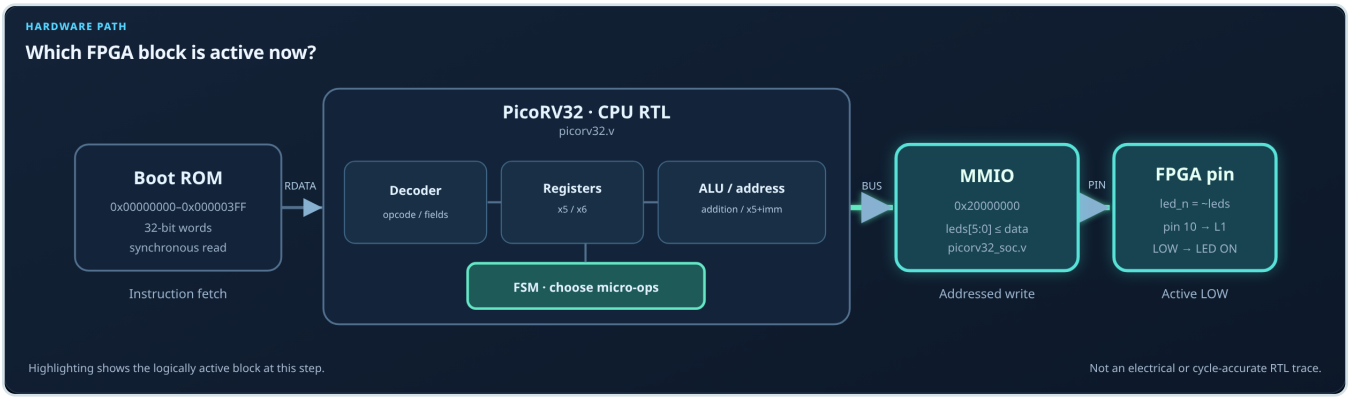
### Where is the actual LED?

`0x20000000` is the **LED register's MMIO address**, not the address of an instruction. SW later stores the value 1 there, leading to `led_n[0]=0` on FPGA pin 10.

> Register-reset caveat: x5 and x6 are **not guaranteed to contain zero** after reset in this PicoRV32 configuration. LUI and ADDI explicitly establish their values. The SoC `leds` register, by contrast, is reset to zero.

HARDWARE DATAPATH

# Where the data goes—and what each block does




English HTML block diagram. Highlighted nodes show logical activity, not precise electrical waveforms.

| Hardware block | Function in the L1 example |
|---|---|
| Boot ROM | Returns instruction words from byte addresses 0x0, 0x4, 0x8, 0xC. |
| PicoRV32 decoder | Recognizes LUI, ADDI, SW and JAL from opcode, funct3 and immediate fields. |
| cpuregs | CPU register x5 holds the address 0x20000000; x6 holds the value 1. |
| ALU and FSM | Perform arithmetic, effective-address formation and control sequencing. |
| Native memory interface | Transfers addresses, data and write strobes; mem_ready acknowledges service. |
| MMIO / leds | The SoC decodes 0x20000000 and captures the low six data bits. |
| FPGA pin 10 / L1 | The combinational inversion led_n[0]=~leds[0] drives LOW and lights L1. |

> There is **no special "turn on LED" instruction** in RISC-V. SW performs an ordinary word store; **our SoC's address decoder** determines which peripheral responds at 0x20000000.

# Instruction fetch at PC=0x00000000

**Student action.** Click **“Step →”** once. The progress indicator should display **“Step 1 of 19”**.

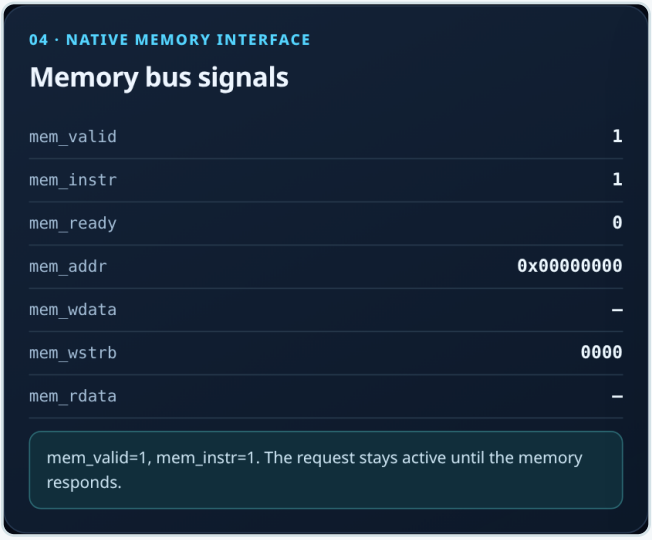
**What the interactive demo shows.** The CPU places address 0x00000000 on the bus and sets mem_instr=1. This reads an instruction from ROM; it does not write the LED register.

## What this means in the FPGA design

The PC is 0 at the start of the fetch. PicoRV32 drives the 32-bit byte address onto its native memory interface and asserts mem_valid. mem_instr=1 identifies an instruction fetch from boot ROM, not an LED-register access. The byte strobes are all zero.

### Values to inspect

| | |
|---|---|
| PC | 0x00000000 |
| mem_valid | 1 |
| mem_instr | 1 |
| mem_ready | 0 |
| mem_addr | 0x00000000 |
| mem_rdata | – |




Memory interface at this step

**Key takeaway:** PC=0x00000000 is the instruction address; 0x20000000 is the LED register address.

**Check your understanding:** Is the CPU addressing program ROM or the LED register at this point?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# ROM returns 0x200002B7

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 2 of 19”**.

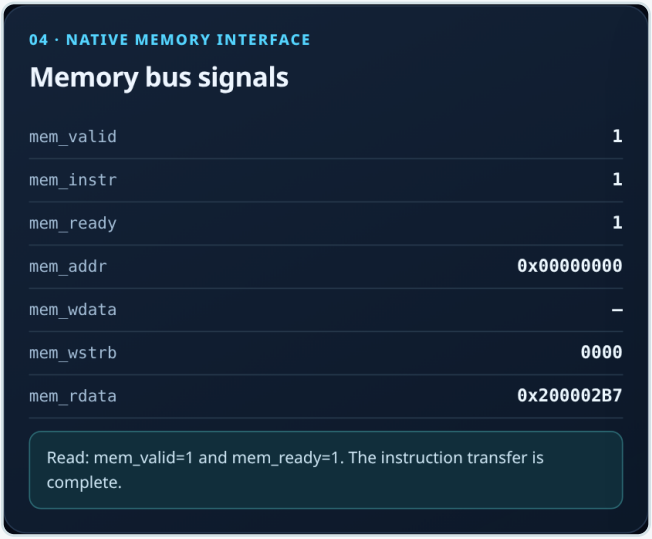
**What the interactive demo shows.** The synchronous ROM selects ROM[0]. With mem_valid and mem_ready high, the CPU accepts 0x200002B7 as an instruction.

## What this means in the FPGA design

The synchronous ROM selects word mem_addr[9:2]=0 and returns 0x200002B7. A transfer is accepted with mem_valid=1 and mem_ready=1. The machine instruction is now available for decoding; x5 and the FPGA pins have not yet changed.

### Values to inspect

| | |
|---|---|
| PC | 0x00000000 |
| mem_valid | 1 |
| mem_instr | 1 |
| mem_ready | 1 |
| mem_addr | 0x00000000 |
| mem_rdata | 0x200002B7 |




Memory interface at this step

**Key takeaway:** The value held in ROM is an instruction encoding, not the MMIO address itself.

**Check your understanding:** What do mem_valid=1 and mem_ready=1 mean together?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# Decoder recognizes LUI

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 3 of 19”**.

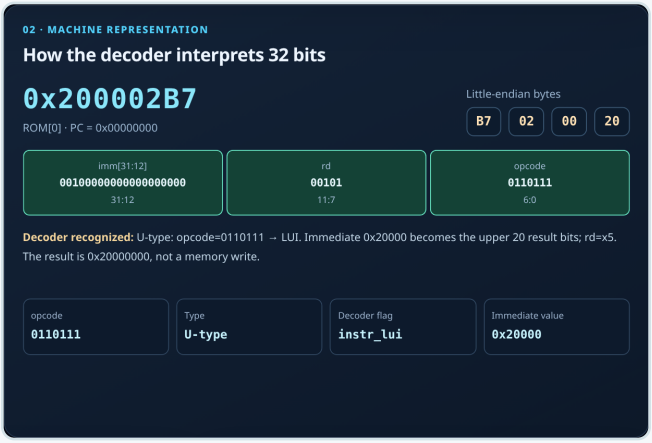
**What the interactive demo shows.** The opcode and additional fields identify the U-type family. The instr_lui flag selects the operation in the control logic.

## What this means in the FPGA design

The decoder reads opcode=0110111 (LUI) from bits [6:0]. Destination rd=00101 means x5; bits [31:12] contain 0x20000. This U-type instruction has no rs1 or rs2 operand. The control logic selects the upper-immediate datapath.

### Values to inspect

| | |
|---|---|
| ROM word | 0x200002B7 |
| opcode | 0110111 |
| Format | U-type |
| Control flag | instr_lui |
| imm | 0x20000 |
| PC | 0x00000000 |




Decoder view of the current instruction

**Key takeaway:** For U-type, the 20-bit immediate is in [31:12] and rd is in [11:7].

**Check your understanding:** Which bits of 0x200002B7 select register x5?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# LUI: build address 0x20000000

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 4 of 19”**.

**What the interactive demo shows.** imm[31:12]=0x20000 is shifted left by 12. The value is written to x5 (t0). The LED is still off.

## What this means in the FPGA design

LUI constructs 0x20000 << 12 = 0x20000000, clears the low 12 bits and writes the result to x5 (t0). The next sequential instruction lies at PC=0x00000004. No MMIO transaction has yet occurred; L1 remains off.

### Values to inspect

| | |
|---|---|
| PC | 0x00000004 |
| x5 (t0) | 0x20000000 |
| x6 (t1) | undefined |
| leds[5:0] | 000000 |
| led_n[5:0] | 111111 |




CPU register file and PC

**Key takeaway:** x5 now contains an address, but the LED has not yet received a write.

**Check your understanding:** Why did writing 0x20000000 to x5 not light L1?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# Instruction fetch at PC=0x00000004

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 5 of 19”**.

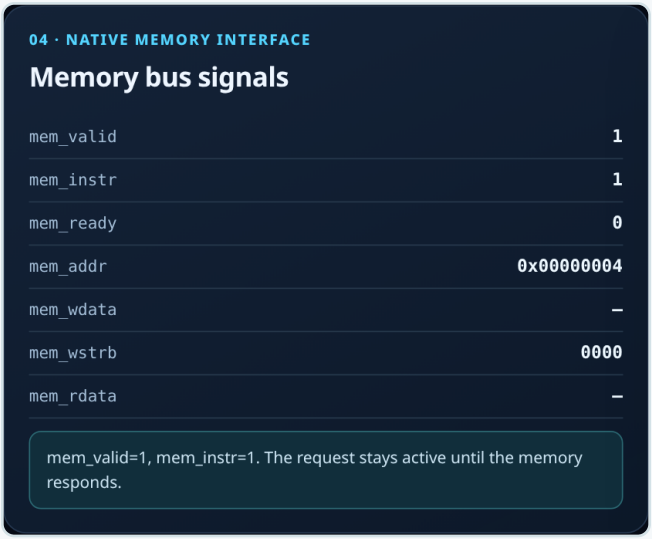
**What the interactive demo shows.** The CPU places address 0x00000004 on the bus and sets mem_instr=1. This reads an instruction from ROM; it does not write the LED register.

## What this means in the FPGA design

The CPU requests ROM[1] at byte address PC=0x00000004. The MMIO address is already in x5, but ADDI does not access the LED register. A new instruction fetch uses the same external memory interface.

### Values to inspect

| | |
|---|---|
| PC | 0x00000004 |
| mem_valid | 1 |
| mem_instr | 1 |
| mem_ready | 0 |
| mem_addr | 0x00000004 |
| mem_rdata | – |




Memory interface at this step

**Key takeaway:** Each instruction is fetched from ROM using the current PC.

**Check your understanding:** Why is the CPU accessing ROM again after x5 has been set?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# ROM returns 0x00100313

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 6 of 19”**.

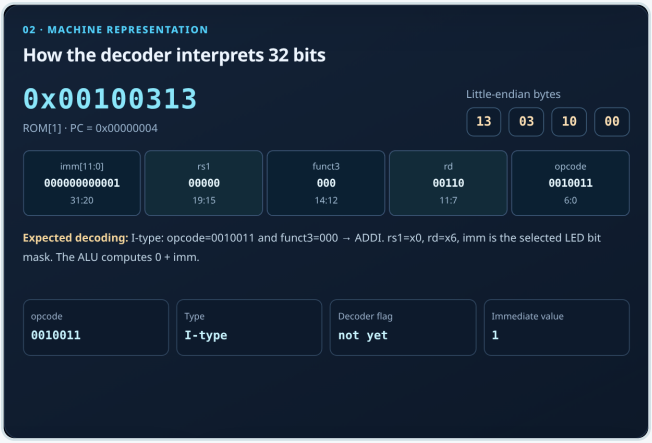
**What the interactive demo shows.** The synchronous ROM selects ROM[1]. With mem_valid and mem_ready high, the CPU accepts 0x00100313 as an instruction.

## What this means in the FPGA design

ROM[1] returns 0x00100313, the machine encoding for ADDI x6,x0,1. The little-endian byte sequence would be 13 03 10 00. The valid/ready handshake completes the instruction fetch before the register writeback stage.

### Values to inspect

| | |
|---|---|
| PC | 0x00000004 |
| mem_valid | 1 |
| mem_instr | 1 |
| mem_ready | 1 |
| mem_addr | 0x00000004 |
| mem_rdata | 0x00100313 |




Memory interface at this step

**Key takeaway:** The little-endian bytes of 0x00100313 are 13 03 10 00.

**Check your understanding:** Which assembly mnemonic corresponds to 0x00100313?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# Decoder recognizes ADDI

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 7 of 19”**.

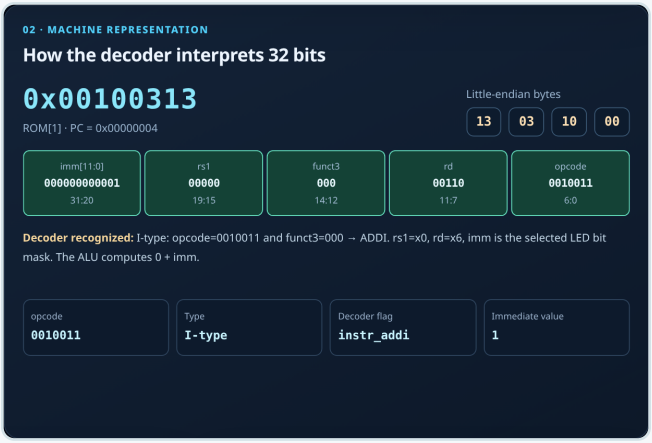
**What the interactive demo shows.** The opcode and additional fields identify the I-type family. The instr_addi flag selects the operation in the control logic.

## What this means in the FPGA design

For I-type arithmetic, opcode=0010011 selects OP-IMM and funct3=000 selects ADDI. rs1=00000 denotes x0, rd=00110 denotes x6, and the signed immediate is 1. The read value of architectural register x0 is always zero.

### Values to inspect

| | |
|---|---|
| ROM word | 0x00100313 |
| opcode | 0010011 |
| Format | I-type |
| Control flag | instr_addi |
| imm | 1 |
| PC | 0x00000004 |




Decoder view of the current instruction

**Key takeaway:** x0 is always read as zero; ADDI computes 0 + 1 here.

**Check your understanding:** How do rs1 and rd differ in the ADDI encoding?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# ADDI: write 1 into x6

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 8 of 19”**.

**What the interactive demo shows.** The ALU adds x0=0 and immediate=1, then writes 0x00000001 to x6 (t1). This will be the data for the LED register.

## What this means in the FPGA design

The arithmetic datapath receives 0 from x0 and the immediate 1. The ALU computes 0x00000001 and writes it to x6 (t1). Both the address (x5) and data (x6) are ready. L1 is still off because SW has not yet written MMIO.

### Values to inspect

| | |
|---|---|
| PC | 0x00000008 |
| x5 (t0) | 0x20000000 |
| x6 (t1) | 0x00000001 |
| leds[5:0] | 000000 |
| led_n[5:0] | 111111 |




CPU register file and PC

**Key takeaway:** x6 holds the future store data while all LEDs remain off.

**Check your understanding:** Which internal CPU register now contains the value 1?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# Instruction fetch at PC=0x00000008

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 9 of 19”**.

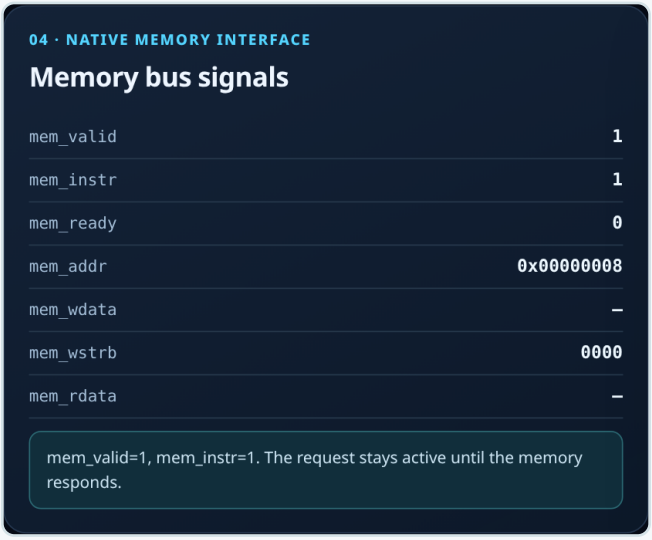
**What the interactive demo shows.** The CPU places address 0x00000008 on the bus and sets mem_instr=1. This reads an instruction from ROM; it does not write the LED register.

## What this means in the FPGA design

The third machine instruction resides at ROM[2], byte address PC=0x00000008. This is an instruction fetch. Do not confuse the instruction address 0x00000008 with the data destination 0x20000000 used later by the same CPU.

### Values to inspect

| | |
|---|---|
| PC | 0x00000008 |
| mem_valid | 1 |
| mem_instr | 1 |
| mem_ready | 0 |
| mem_addr | 0x00000008 |
| mem_rdata | – |




Memory interface at this step

**Key takeaway:** At this stage mem_addr=0x00000008 is an instruction address.

**Check your understanding:** Why must we distinguish 0x00000008 from 0x20000000?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# ROM returns 0x0062A023

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 10 of 19”**.

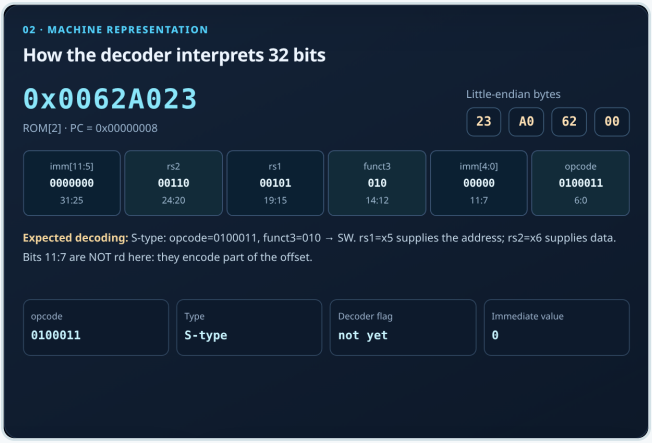
**What the interactive demo shows.** The synchronous ROM selects ROM[2]. With mem_valid and mem_ready high, the CPU accepts 0x0062A023 as an instruction.

## What this means in the FPGA design

ROM[2] returns 0x0062A023, which means SW x6,0(x5). The word's little-endian bytes would be 23 A0 62 00. The bits are still only a fetched instruction; the decoder must first interpret them.

### Values to inspect

| | |
|---|---|
| PC | 0x00000008 |
| mem_valid | 1 |
| mem_instr | 1 |
| mem_ready | 1 |
| mem_addr | 0x00000008 |
| mem_rdata | 0x0062A023 |




Memory interface at this step

**Key takeaway:** The 0x0062A023 bits alone cannot light an LED until SW is executed.

**Check your understanding:** What are the little-endian bytes of the SW machine word?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# Decoder recognizes SW

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 11 of 19”**.

**What the interactive demo shows.** The opcode and additional fields identify the S-type family. The instr_sw flag selects the operation in the control logic.

## What this means in the FPGA design

In the S format, opcode=0100011 means STORE and funct3=010 identifies SW, a full 32-bit word store. rs1=00101 selects x5, rs2=00110 selects x6. The immediate is composed from fields [31:25] and [11:7] and is zero here. SW has no rd destination.

### Values to inspect

| | |
|---|---|
| ROM word | 0x0062A023 |
| opcode | 0100011 |
| Format | S-type |
| Control flag | instr_sw |
| imm | 0 |
| PC | 0x00000008 |




Decoder view of the current instruction

**Key takeaway:** S-type bits [11:7] are part of the immediate, not an rd field.

**Check your understanding:** Why is there no rd field in an S-type instruction?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# SW: where do address and data come from?

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 12 of 19”**.

**What the interactive demo shows.** The decoder extracts rs1=5 (x5), rs2=6 (x6), imm=0. The register file returns x5=0x20000000 and x6=0x00000001. The address datapath computes x5+0.

## What this means in the FPGA design

The register file supplies x5=0x20000000 as the base address and x6=0x00000001 as the write data. The address datapath computes x5 + sign_extend(0). In the actual core, the STORE path uses cpu_state_stmem; the HTML highlights address preparation separately from bus acknowledgment.

### Values to inspect

| | |
|---|---|
| PC | 0x00000008 |
| x5 (t0) | 0x20000000 |
| x6 (t1) | 0x00000001 |
| leds[5:0] | 000000 |
| led_n[5:0] | 111111 |




CPU register file and PC

**Key takeaway:** Decode selects SW; the address datapath then forms x5 + 0.

**Check your understanding:** Where do the effective address and write data for SW come from?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# SW writes to address 0x20000000

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 13 of 19”**.

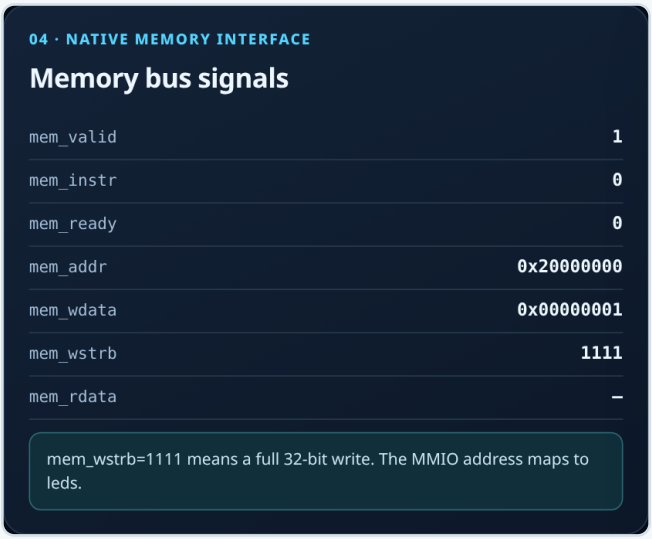
**What the interactive demo shows.** PicoRV32 drives mem_valid=1, mem_instr=0, mem_wstrb=1111, mem_addr=0x20000000 and mem_wdata=0x00000001. MMIO decodes the address and selects the LED register.

## What this means in the FPGA design

PicoRV32 presents mem_valid=1, mem_instr=0, mem_addr=0x20000000, mem_wdata=0x00000001 and mem_wstrb=1111. The SoC address decoder selects the MMIO region and its LED register. With mem_ready=0 the transfer is not yet acknowledged, so leds has not changed in the displayed model.

### Values to inspect

| | |
|---|---|
| mem_valid | 1 |
| mem_ready | 0 |
| mem_instr | 0 |
| mem_addr | 0x20000000 |
| mem_wdata | 0x00000001 |
| mem_wstrb | 1111 |
| leds[5:0] | 000000 |




Current system state

**Key takeaway:** With mem_ready still low, the modeled LED register remains unchanged.

**Check your understanding:** What if mem_ready remains low indefinitely?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# L1 turns ON: FPGA pin 10 goes LOW

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 14 of 19”**.

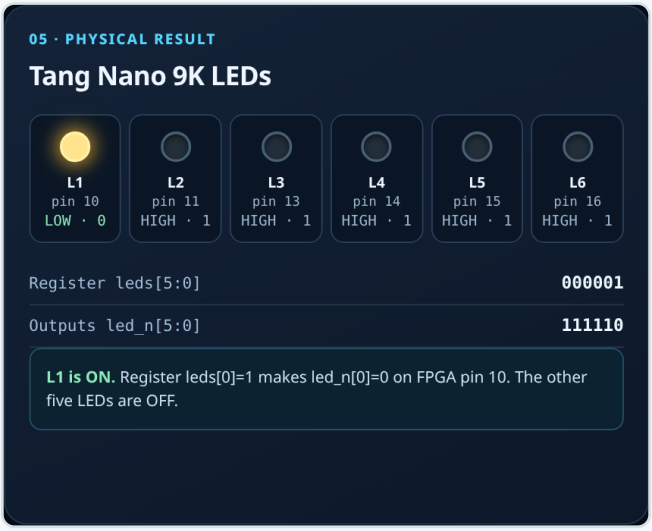
**What the interactive demo shows.** On the active clock edge the SoC writes 000001 into leds[5:0]. The expression led_n[0]=~leds[0] makes the output 0. mem_ready=1 acknowledges SW.

## What this means in the FPGA design

On an active FPGA clock edge, the SoC executes leds <= mem_wdata[5:0], setting leds=000001. In that clocked block it also registers mem_ready=1. The combinational assignment led_n=~leds then produces 111110, so FPGA pin 10 is LOW and L1 lights. The CPU completes the handshake when valid and ready coincide on a subsequent relevant clock edge.

### Values to inspect

| | |
|---|---|
| mem_valid | 1 |
| mem_ready | 1 |
| mem_instr | 0 |
| mem_addr | 0x20000000 |
| mem_wdata | 0x00000001 |
| mem_wstrb | 1111 |
| leds[5:0] | 000001 |




LED register and physical FPGA pin states

**Key takeaway:** The logical leds=000001 is inverted to physical led_n=111110.

**Check your understanding:** Why does leds[0]=1 make the output on pin 10 LOW?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# Store complete; L1 stays ON

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 15 of 19”**.

**What the interactive demo shows.** The CPU releases the request and proceeds to PC=0x0000000C. The LED register retains 1; the lamp does not depend on later CPU instructions.

## What this means in the FPGA design

After the acknowledged SW transaction the CPU deasserts its request and proceeds to PC=0x0000000C. The six flip-flops making up the LEDs register retain 000001. Its output is independent of subsequent instructions until another MMIO write or a reset changes it.

### Values to inspect

| | |
|---|---|
| PC | 0x0000000C |
| x5 (t0) | 0x20000000 |
| x6 (t1) | 0x00000001 |
| leds[5:0] | 000001 |
| led_n[5:0] | 111110 |




LED register and physical FPGA pin states

**Key takeaway:** MMIO flip-flops preserve LED state independently of later PC changes.

**Check your understanding:** Why does the LED stay on without repeating SW?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# Instruction fetch at PC=0x0000000C

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 16 of 19”**.

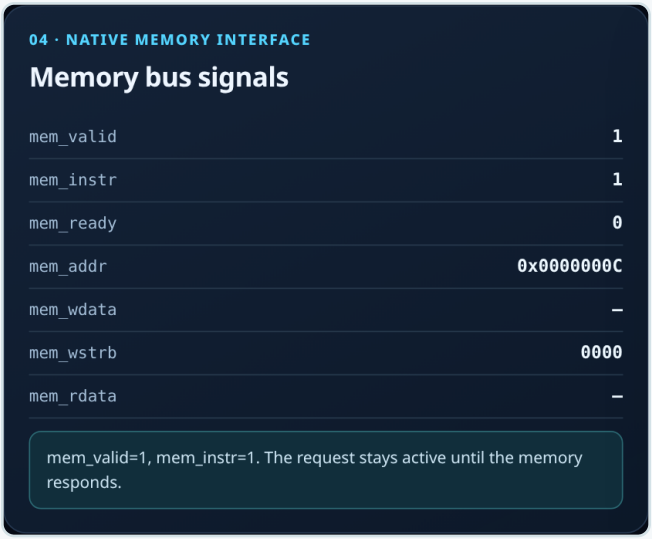
**What the interactive demo shows.** The CPU places address 0x0000000C on the bus and sets mem_instr=1. This reads an instruction from ROM; it does not write the LED register.

## What this means in the FPGA design

The CPU now fetches ROM[3] at PC=0x0000000C. Instruction fetch does not target the LED MMIO register. The previously written GPIO output remains asserted even though the CPU is busy doing something else.

### Values to inspect

| | |
|---|---|
| PC | 0x0000000C |
| mem_valid | 1 |
| mem_instr | 1 |
| mem_ready | 0 |
| mem_addr | 0x0000000C |
| mem_rdata | – |




Memory interface at this step

**Key takeaway:** Fetching JAL does not change the peripheral register.

**Check your understanding:** How does fetching an instruction differ from reading MMIO data?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# ROM returns 0x0000006F

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 17 of 19”**.

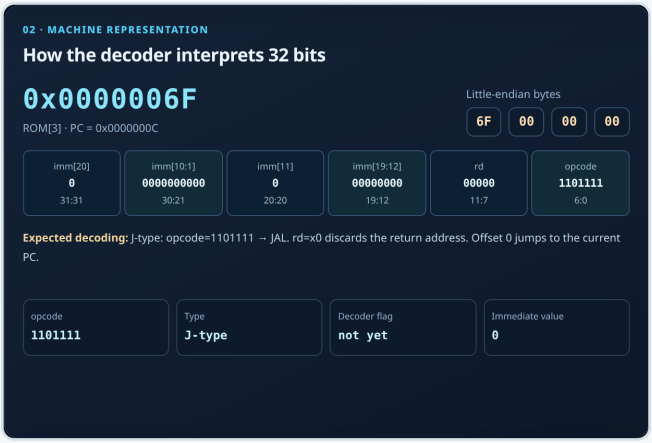
**What the interactive demo shows.** The synchronous ROM selects ROM[3]. With mem_valid and mem_ready high, the CPU accepts 0x0000006F as an instruction.

## What this means in the FPGA design

ROM[3] returns 0x0000006F, the encoding of JAL x0,0. As for previous fetches, the valid/ready handshake transfers the instruction to the core; the stored LED value is unaffected.

### Values to inspect

| | |
|---|---|
| PC | 0x0000000C |
| mem_valid | 1 |
| mem_instr | 1 |
| mem_ready | 1 |
| mem_addr | 0x0000000C |
| mem_rdata | 0x0000006F |




Memory interface at this step

**Key takeaway:** 0x0000006F encodes JAL x0,0 at PC=0x0000000C.

**Check your understanding:** What is the machine code for JAL with offset zero?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# Decoder recognizes JAL

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 18 of 19”**.

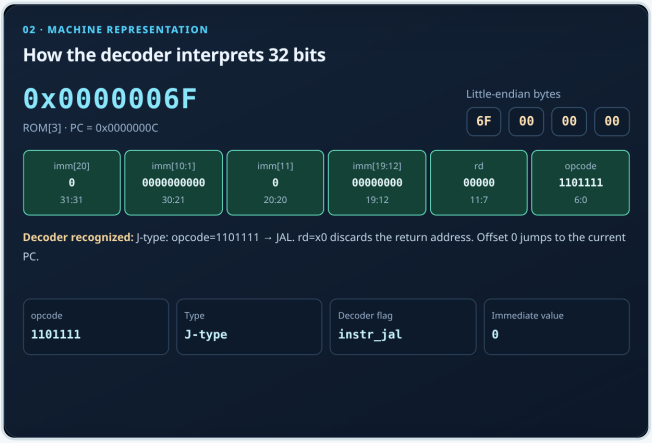
**What the interactive demo shows.** The opcode and additional fields identify the J-type family. The instr_jal flag selects the operation in the control logic.

## What this means in the FPGA design

opcode=1101111 identifies JAL. rd=00000 selects x0, so the return address write is discarded. The J-type immediate is assembled from scattered bit fields and equals zero here. This instruction does not perform an MMIO read or write.

### Values to inspect

| | |
|---|---|
| ROM word | 0x0000006F |
| opcode | 1101111 |
| Format | J-type |
| Control flag | instr_jal |
| imm | 0 |
| PC | 0x0000000C |




Decoder view of the current instruction

**Key takeaway:** Using rd=x0 discards the return address of JAL.

**Check your understanding:** Where would JAL save its return address if rd were not x0?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

# The CPU repeats JAL while L1 stays ON

**Student action.** Click **“Step →”** again. The progress indicator should display **“Step 19 of 19”**.

**What the interactive demo shows.** JAL x0,0 has an offset of 0: PC remains 0x0000000C. The core continues fetching and executing this instruction forever. The LED stays on because leds[0] is not overwritten.

## What this means in the FPGA design

JAL computes the next PC as 0x0000000C + 0, returning to itself indefinitely. The real CPU keeps fetching and executing JAL; the HTML visualization stops after demonstrating the first loop. L1 stays on because the leds register is never rewritten.

### Values to inspect

| | |
|---|---|
| PC | 0x0000000C |
| x5 (t0) | 0x20000000 |
| x6 (t1) | 0x00000001 |
| leds[5:0] | 000001 |
| led_n[5:0] | 111110 |




LED register and physical FPGA pin states

**Key takeaway:** The JAL loop does not repeat the LED-store instruction.

**Check your understanding:** Why does the infinite JAL loop leave L1 illuminated?

This page describes one conceptual step in the HTML model, not necessarily one clock cycle in the real RTL, nor a measured gate-level timing event.

INDEPENDENT EXPLORATION

# Change the target: from L1 to L2–L6

Choose another LED at the top of the interactive page. Only the constant in ADDI changes; the MMIO base and SW machine word remain unchanged. For example, L4 uses bit mask 8, so x6=8 and physical FPGA pin 14 goes LOW.

| LED | FPGA pin | ADDI word | x6 | leds[5:0] | led_n[5:0] |
|---|---|---|---|---|---|
| L1 | `pin 10` | 00100313 | 0x00000001 | 000001 | 111110 |
| L2 | `pin 11` | 00200313 | 0x00000002 | 000010 | 111101 |
| L3 | `pin 13` | 00400313 | 0x00000004 | 000100 | 111011 |
| L4 | `pin 14` | 00800313 | 0x00000008 | 001000 | 110111 |
| L5 | `pin 15` | 01000313 | 0x00000010 | 010000 | 101111 |
| L6 | `pin 16` | 02000313 | 0x00000020 | 100000 | 011111 |

## Student exercises

1. Press “↻ Reset” and select L4. Note that x6 is not yet defined.
2. Go to step 8. Verify x6=0x00000008 and ADDI machine word 0x00800313.
3. Go to step 13. Check mem_wdata=0x00000008 while mem_ready is still 0.
4. Go to step 14. Verify leds=001000, led_n=110111, FPGA pin 14 LOW.
5. Explain why neither x5 nor x6 needs to store physical pin number 14 as a memory address.

> Selecting a new LED in the browser changes the **educational model only**; it does not reconfigure a real FPGA. A board experiment needs regenerated ROM content, a rebuilt .fs and RTL/board verification.

ENGINEERING REFERENCE

# How the HTML relates to real Verilog

## 1. Synchronous boot ROM

```
boot_rom u_rom (.clk(clk), .word_addr(mem_addr[9:2]), .rdata(rom_data));
// Educational ROM[0]=32'h200002B7, ROM[1]=32'h00100313, ...
```

The four-instruction LED example differs from our physically tested 33-instruction firmware. Both use the same ROM-addressing scheme and SoC MMIO mapping.

## 2. Decoder fields and STORE selection

```
decoded_rs1 <= mem_rdata_latched[19:15];
decoded_rs2 <= mem_rdata_latched[24:20];
// For SW: opcode = 7'b0100011, funct3 = 3'b010.
```

## 3. MMIO acknowledgment and LED output

```
wire is_mmio = (mem_addr[31:8] == 24'h200000);
// For MMIO address 0x20000000, inside the clocked block:
if (|mem_wstrb && mem_wstrb[0]) leds <= mem_wdata[5:0];
mem_ready <= 1'b1;
assign led_n = ~leds;
```

## Three essential distinctions

1. **Conceptual model ≠ RTL simulator.** Nineteen steps are not nineteen clock cycles. The HTML does not run Verilog.
2. **The onscreen IR is an educational view of the fetched instruction.** It does not prove a separately named RTL register exists inside this revision of the CPU core.
3. **Hardware validation ≠ validation of this new program.** On 19 October 2026, the 33-instruction firmware passed the Tang Nano 9K test (UART “PICO” and cycling LEDs). The four-instruction LED-only example is not independently hardware-verified.

ASSESSMENT

# Review questions and primary sources

## Questions

1. Where is instruction word 0x200002B7 stored?
2. Why does x5 equal 0x20000000 after LUI?
3. How does mem_addr differ at steps 9 and 13?
4. Which fields identify SW in 0x0062A023?
5. When is a memory transaction considered complete?
6. Why does leds[0]=1 produce LOW on FPGA pin 10?
7. Why does L1 remain on during repeated JAL?

## Lab report checklist

1. Identify your selected LED and physical pin.
2. List the four ROM words with addresses.
3. Show x5/x6 values after step 8.
4. Copy memory-interface values at steps 13 and 14.
5. Show the logical inversion leds → led_n.
6. State the limitations of an HTML-level model.

## Project and source code

**Project:** github.com/AIDevelopersMonster/Sipeed-FPGA
**Original core:** github.com/YosysHQ/picorv32
**SoC sources:** `01-PicoRV32/rtl/picorv32_soc.v`, `boot_rom.v`, `constraints/tang_nano_9k.cst`.

**Attribution:** this HTML educational demonstration and guide were authored by **A. A. Malachevsky**, Independent Researcher, ORCID 0009-0008-6009-3196. The original PicoRV32 core is © Claire Xenia Wolf, distributed under the ISC license. The educational integration does not claim authorship of the upstream processor.